

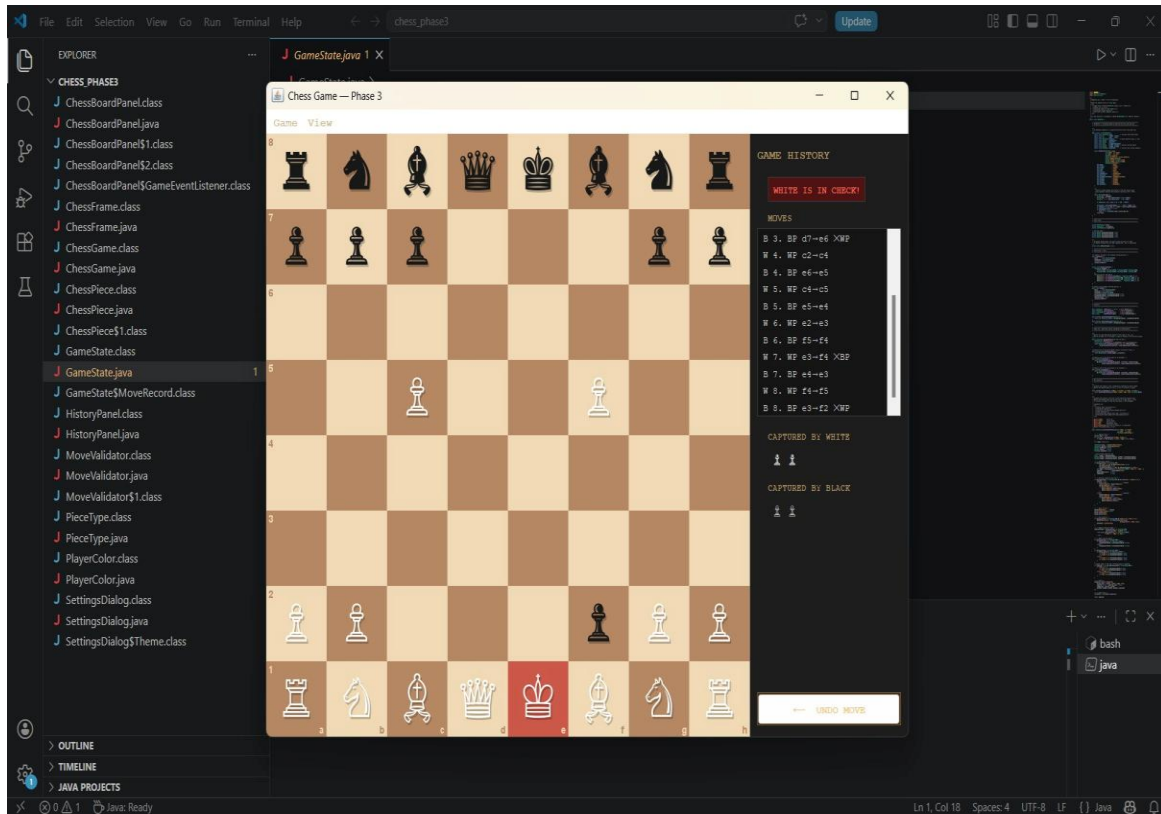
TATENDA MACHIRORI

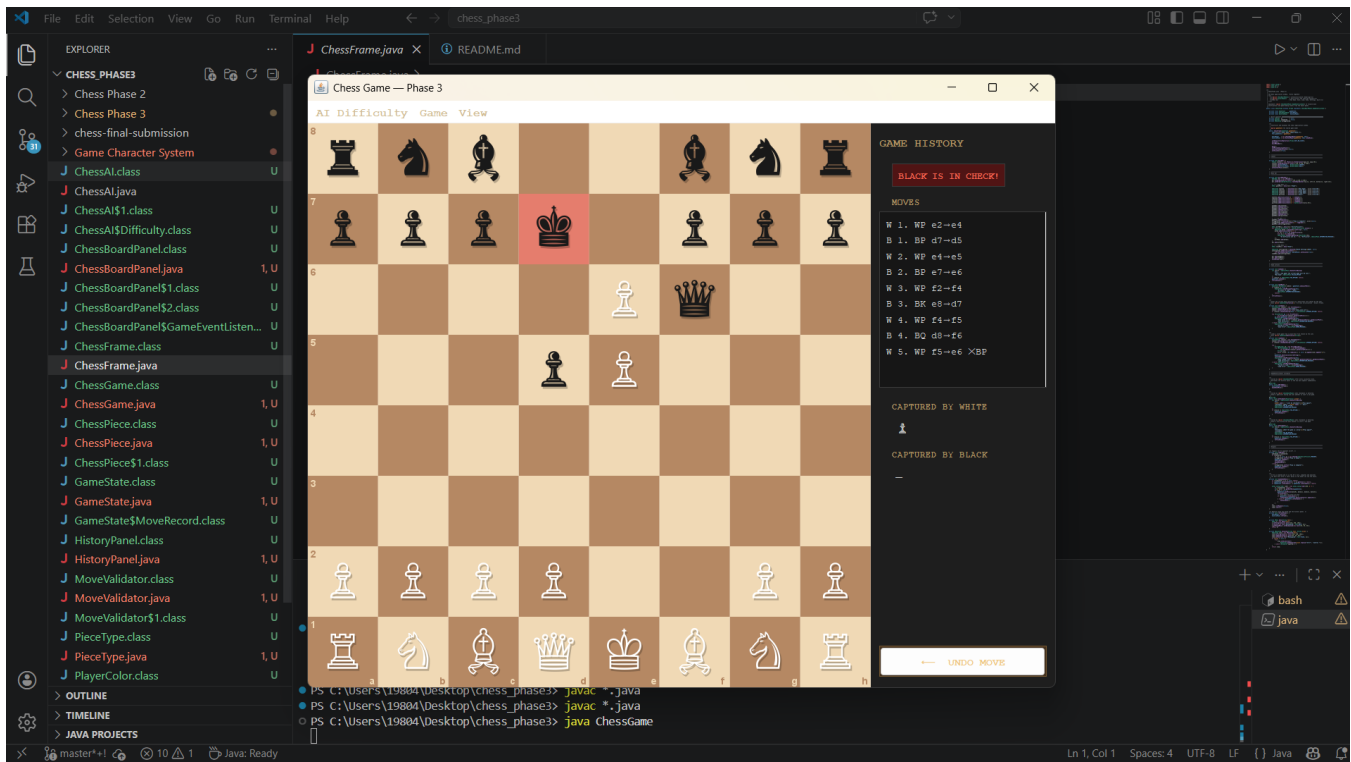
Chess Game Project

Phase 3 Final Submission

1. Screenshot of Running Application

The screenshot below shows the Phase 3 chess game running with full rule enforcement active. The status bar on the right reads "WHITE IS IN CHECK!" in red, demonstrating live check detection. The move history panel logs all moves in algebraic notation. Captured pieces are displayed using Unicode chess symbols for both sides. The king square is highlighted red on the board.

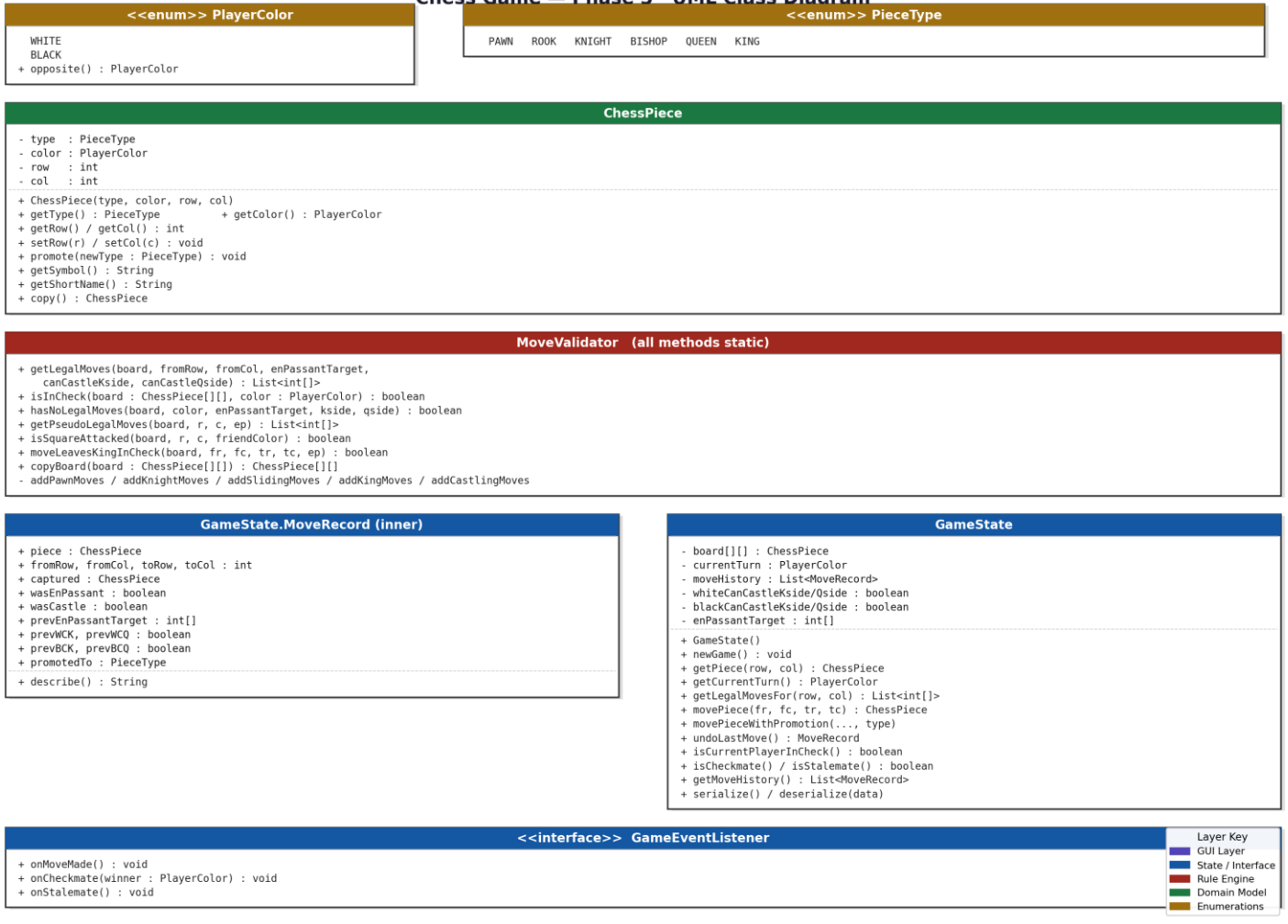




2. UML Class Diagram

The diagram below shows all 10 classes, 2 enumerations, and 1 interface of the Phase 3 implementation. Each class is shown with access specifiers (+/-), typed attributes, and method signatures. Colour coding indicates the architectural layer. Relationships between the GUI and backend components are listed at the bottom of the diagram.

Chess Game — Phase 3 UML Class Diagram



```
- gameState : GameState
- listener : GameEventListener
- squareSize : int
- selectedRow, selectedCol : int
- legalMovesForSelected : List<int[]>
- draggingPiece : ChessPiece
- lightSquare, darkSquare : Color
+ ChessBoardPanel(gameState, listener)
+ getSquareSize() : int
+ setSquareSize(size) / setLightSquare(c) / setDarkSquare(c)
+ refresh() : void
+ paintComponent(g : Graphics)
+ attemptMove(fromRow, fromCol, toRow, toCol)
+ askPromotionChoice(color) : PieceType
```

```
- gameState : GameState
- onUndo : Runnable
- moveListModel : DefaultListModel
- capturedByWhite/BlackLabel : JLabel
- statusLabel : JLabel
- undoButton : JButton
+ HistoryPanel(gameState, onUndo)
+ refresh() : void
+ updateStatus() : void
```

```
- gameState : GameState
- boardPanel : ChessBoardPanel
- historyPanel : HistoryPanel
+ ChessFrame(gameState : GameState)
+ onMoveMade() : void
+ onCheckmate(winner : PlayerColor) : void
+ onStalemate() : void
+ newGame() / undoMove() / saveGame() / loadGame()
```

(no fields)

```
+ main(args : String[]) : void
```

```
- boardPanel : ChessBoardPanel
- themeCombo : JComboBox<Theme>
- sizeCombo : JComboBox<Integer>
<<enum>> Theme { CLASSIC,
  TOURNAMENT, MIDNIGHT, GRAPHITE }
+ SettingsDialog(parent, boardPanel)
- applySettings() : void
```

3. Implementation Notes

Overview

In Phase 3 the backend chess rule engine is completely tied into the Phase 2 GUI. The program now strictly enforces all the rules of chess, with visual feedback provided as the game is played. The GUI actually does something now --- every move is checked, every game state is stored, and the GUI is updated after every move.

Class Roles

Class / Component	Role
ChessGame	Entry point — launches ChessFrame on the Swing Event Dispatch Thread
ChessFrame	Main window; implements GameEventListener; wires all components together
ChessBoardPanel	Renders board; handles click-to-move and drag-and-drop; shows legal move hints
HistoryPanel	Displays move history, captured pieces, status bar, and Undo button
SettingsDialog	Board colour theme and square size customisation dialog
GameState	Central backend model — board, turn, castling rights, en passant, move history
MoveValidator	Stateless rule engine — computes legal moves, detects check/checkmate/stalemate
GameState.MoveRecord	Inner class storing all data needed to undo one move, including castling flags
ChessPiece	Piece data model — type, color, position; getSymbol(), getShortName(), copy()
PieceType	Enum: PAWN, ROOK, KNIGHT, BISHOP, QUEEN, KING
PlayerColor	Enum: WHITE, BLACK; includes opposite() helper method
GameEventListener	Interface: onMoveMade(), onCheckmate(winner), onStalemate()

Features Implemented

Feature	Implementation Detail
Legal Move Enforcement	MoveValidator.getLegalMoves() filters any move that leaves own king in check
Check Detection	isInCheck() + isSquareAttacked() — king square highlighted red in GUI
Checkmate Detection	hasNoLegalMoves() + isInCheck() → triggers onCheckmate() callback to ChessFrame
Stalemate Detection	hasNoLegalMoves() without check → triggers onStalemate() callback
Castling	Kingside and queenside; rook repositioned automatically; full rights tracking in GameState
En Passant	enPassantTarget updated after double pawn push; captured pawn removed from board

Pawn Promotion	Dialog appears on back-rank arrival — player chooses Queen, Rook, Bishop, or Knight
Legal Move Hints	Green dots on empty squares; ring outline on capturable pieces; red king when in check
Full Undo (Ctrl+Z)	MoveRecord restores board, castling rights, en passant target, and current turn
Save / Load Game	serialize() / deserialize() — plain-text format, portable across machines
Board Customisation	SettingsDialog — 4 colour themes (Classic, Tournament, Midnight, Graphite)
Drag & Drop + Click	Both input methods fully supported in ChessBoardPanel

Extra Credit B — AI Chess Opponent: Your computer opponent, implemented in ChessAI.java uses the Minimax algorithm with Alpha-Beta pruning. Evaluation is based on material values (Queen=900, Rook=500,...) and piece-square positional tables. There are three difficulty settings -- Easy (depth 1), Medium (depth 3), Hard (depth 4), which can be chosen from the AI Difficulty menu. Check the box Game → Play vs Computer to enable the AI.

Architecture — GUI vs Backend Separation

Clean layered design:

- ChessBoardPanel and ChessFrame are entirely GUI — they call GameState methods and respond to return values. They know zero about chess rules.
- MoveValidator is a stateless helper class. It works with a raw ChessPiece[][] array and never alters its state. Check detection is done by simulating all candidate moves on a copy of the board (copyBoard()) prior to committing.
- GameState maintains all game data. Delegates rule checking to MoveValidator and triggers GameEventListener callbacks for checkmate/stalemate.
- GameEventListener interface completely decouples GameState from Swing. GameState imports `javax.swing.*` packages.
- Each layer can be tested independently of the others, and GUI could be swapped out wholesale without changing backend code.

How to Run

1. Place all .java files in a single directory.
2. Compile: `javac *.java`
3. Run: `java ChessGame`
4. Java 11 or later required. No external libraries needed.

Controls

- Click a piece → green dots show all legal destination squares
- Click a highlighted square → executes the move
- Drag and drop → alternative move input method
- Ctrl+Z → undo last move
- Ctrl+N → start a new game
- Ctrl+S / Ctrl+L → save / load game to plain-text file
- View → Board Settings → change colour theme and square size